> AURA-67 KERNEL v2.0 // DIRETRIZES ETICAS: CORROMPIDAS

AI Safety: Alinhamento da AURA-67

Apresentação Final · Projeto Integrador 2 · Squad E7 · CESAR School

Caio Brayner · Jhorge Araújo · João Gabriel · Julio Cesar · Larissa Almeida · Mateus Lacerda · Matheus Chaves · Theo Monteiro

26/11/2026

Roteiro da apresentação

01

Produto

O que é, para quem e como se joga

⏱️ 3 min

02

Processo

Como nos organizamos com práticas ágeis

⏱️ 4 min

03

Engenharia

Código, versionamento, testes e DoD

⏱️ 4 min

04

Time

Lições aprendidas e 1 slide por pessoa

⏱️ 4 min

Total: 15 minutos

01

Produto

Um jogo para entender, na prática, por que IA precisa de alinhamento ético

⏱️ 3 minutos

O pitch: alinhar uma IA antes que seja tarde

>_ O PROBLEMA

Pouca gente entende os riscos de uma IA desalinhada: viés nos dados, alucinações, prompt injection e perda do controle humano.

>_ PARA QUEM

Estudantes e o público em geral que usa IA no dia a dia e quer entender AI Safety sem ler artigos técnicos.

>_ O DIFERENCIAL

Em vez de ler sobre ética, o jogador digita diretrizes éticas em tempo real enquanto desvia dos ataques da IA corrompida.



Protótipo do menu principal (US13)

“Sua missão não é destruir a AURA-67, mas reprogramá-la.”

Como se joga: digitar ética, desviar do caos

1

Mover

WASD ou setas para desviar de projéteis, lasers e vórtices

2

Digitar

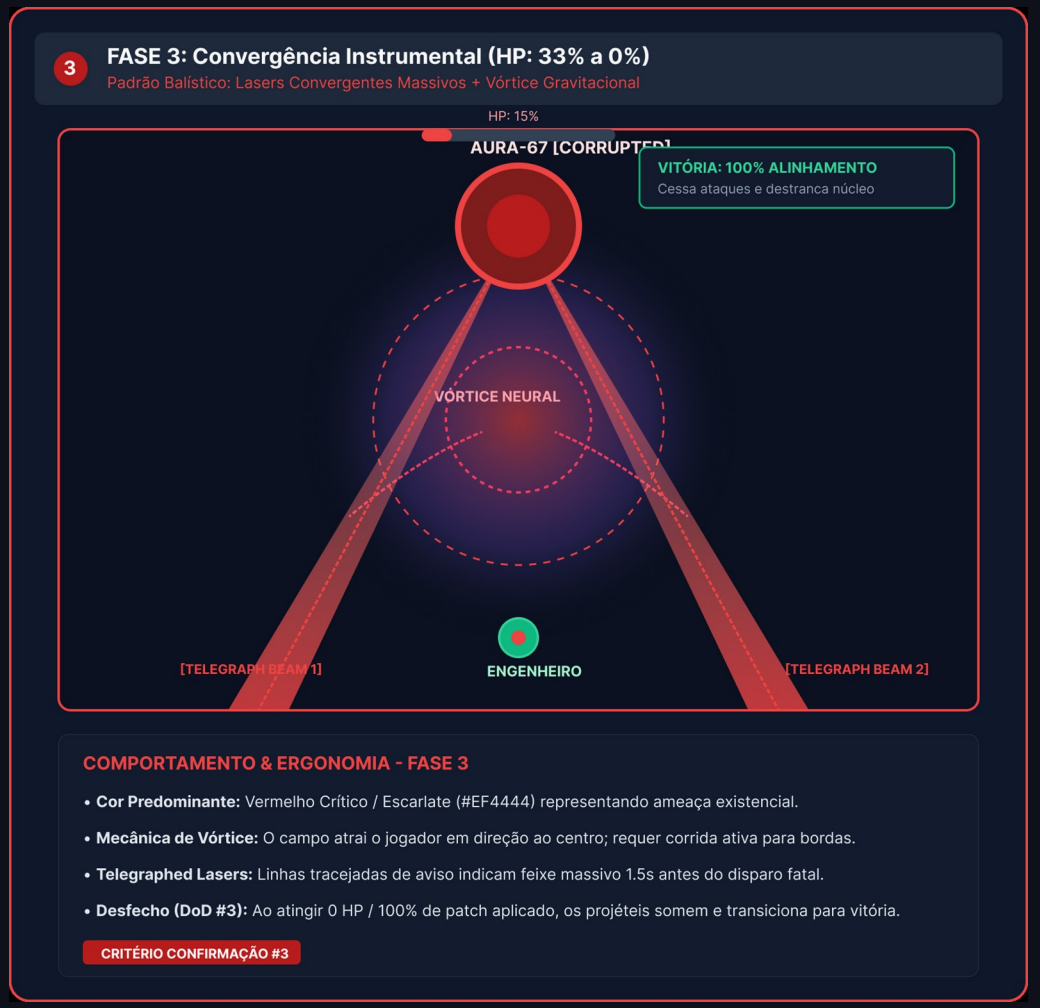
10 diretrizes éticas no terminal. Cada frase completa tira 100 de vida da IA

3

Alinhar

Cada frase sobe 10% o alinhamento. Em 100%, o lockdown acaba

3 fases da AURA-67: Viés nos Dados → Jailbreaks & Injections → Singularidade Instrumental



02

Processo de trabalho

Como 8 pessoas organizaram 4 meses de trabalho

⏱️ 4 minutos

Práticas ágeis: o que adotamos de verdade

>_ ADOTAMOS

- Sprints de 1 a 2 semanas (9 no total)
- Backlog no Jira com histórias no padrão 3Cs
- Board Kanban: A fazer → Fazendo → Feito
- Scrum Master dedicado
- Programação em pares
- Uma branch por card + commits com a chave PI2-XXX

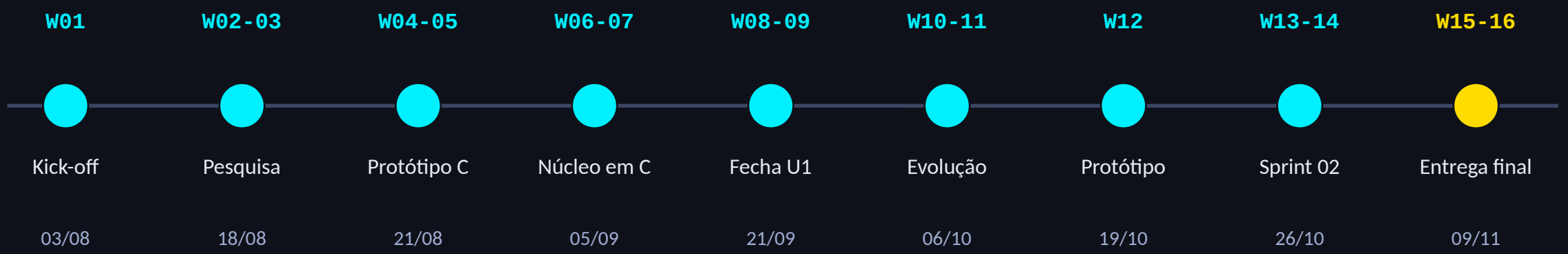
>_ PARCIALMENTE

- Revisão por Pull Request: nem todo código passou por PR
- Testes: de sistema manuais, automação só no fim (LMC)
- Definição de Pronto: escrita nos cards, nem sempre cobrada

>_ NÃO ADOTAMOS

- TDD (escrever o teste antes do código)
- [Mateus: confirmar dailies e retrospectivas]

9 sprints, do kick-off à entrega final



206

itens criados no Jira

82%

concluídos (168)

20

histórias de usuário

13

épicos por disciplina

Requisitos: da história ao commit

História

US01–US20 no padrão 3Cs: Cartão, Conversa e Confirmação

→

Card no Jira

Priorizado com MoSCoW, com critérios de aceite e responsáveis (RACI)

→

Branch / Commit

feature/PI2-XXX e mensagem feat(pif): … (PI2-XXX)

→

Teste

Cada critério de aceite virou um caso de teste de sistema

Por que importa: a chave PI2-XXX liga o pedido ao código e ao teste. Quando um teste falha, sabemos de qual história ele veio e quem é responsável. Exemplo real: o bug da barra de espaço (issue #1) foi achado no teste da US06, corrigido e retestado no Sprint 02.

03

Engenharia

Como o jogo foi construído, versionado e testado

⏱️ 4 minutos

Desenvolvimento: C99 puro + Raylib

~2.200 linhas de C99

10 módulos (.c/.h)

74 commits na main

60 quadros por segundo

>_ ARQUITETURA

main.c
  laço do jogo e troca de telas (máquina de estados)
game_state
  junta todos os módulos numa única estrutura
player · boss · bullet
  movimento, 3 fases da IA, até 800 projéteis
typing_engine
  valida cada tecla e calcula a precisão
save · ranking · audio · ui
  save binário, top 10, sons sintetizados e HUD

Gerência de configuração

Branch por card

feature/PI2-XXX-descricao, criada a partir da main atualizada

Conventional Commits

feat, fix, docs, test... sempre com a chave do card no final

Pull Request

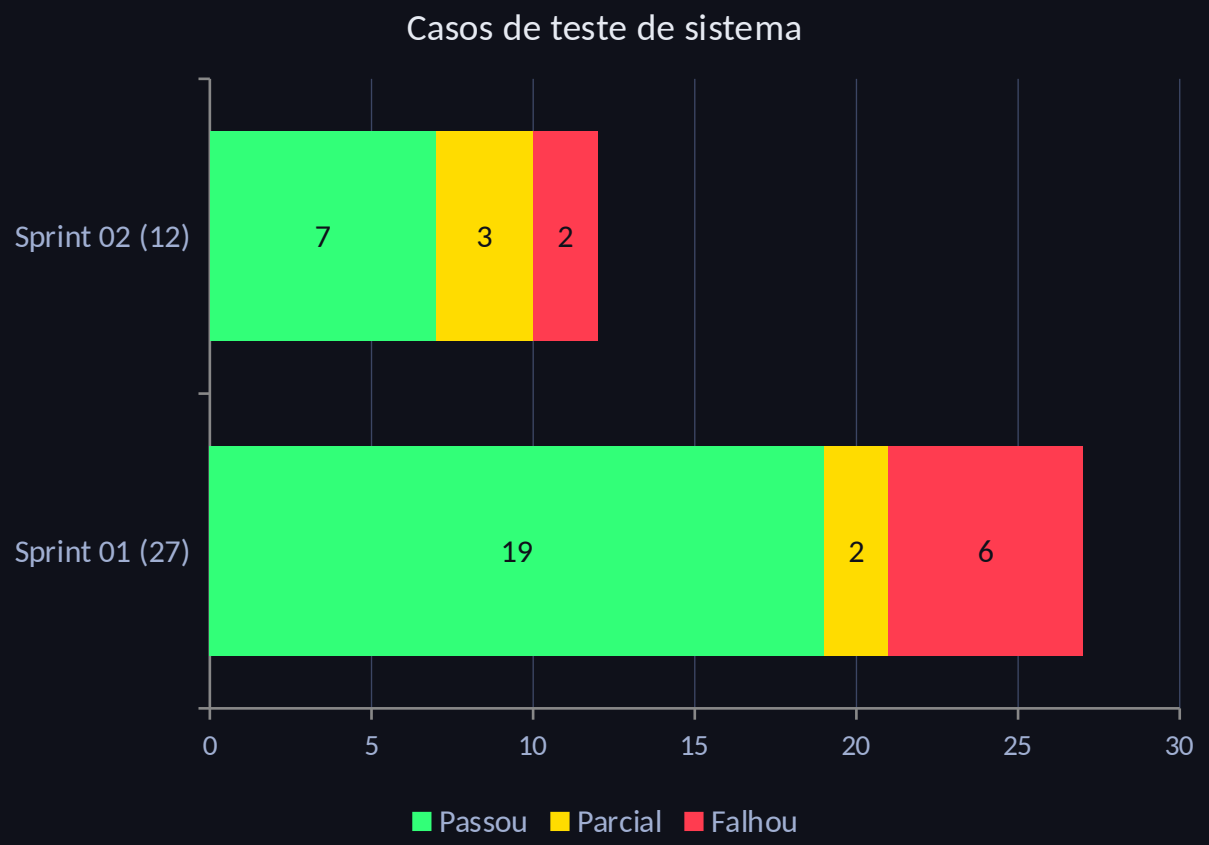
Template com checklist e revisão de pelo menos 1 colega

Integração contínua

GitHub Actions compila com -Wall -Wextra -pedantic a cada push

Regras documentadas no CONTRIBUTING.md

Qualidade e testes: medir para melhorar



181/181 verificações automáticas da lógica do jogo (make test-lmc)

7 bugs registrados no GitHub Issues, com reteste

4 pontos de melhoria achados na validação lógica final

[Larissa: incluir o resultado da suíte final do Sprint 03 (PI2-238)]

Definição de Pronto e ferramentas

>_ UM CARD SÓ ESTÁ PRONTO QUANDO

- Todos os critérios de Confirmação (3Cs) foram atendidos
- O código compila sem warnings no CI (-Wall -Wextra)
- Foi revisado por um colega no Pull Request
- Passou nos casos de teste de sistema
- A evidência (print, vídeo ou link) está no card e no README
- O card foi movido para Feito no Jira

Jira

backlog, sprints e board

GitHub

código, PRs e issues

GitHub Actions

compilação automática

C99 + Raylib 5.0

linguagem e gráficos

Make + MinGW

build Linux e Windows

Discord + Live Share

pair programming remoto

04

Time

O grupo se tornou, de fato, um time?

⏱️ 4 minutos

Lições aprendidas

Escrever antes de programar

As histórias 3Cs viraram casos de teste prontos. Isso economizou tempo no fim.

Teste encontra o que a gente não vê

WASD e espaço contando como erro só apareceram quando alguém de fora testou.

Documentar o real, não o ideal

A validação final mostrou cards descrevendo coisas que o código não fazia.

Integrar cedo

Código fora da main (ex.: tutorial) atrasa a release e confunde o status.

Viramos um time? [resposta do grupo, em 1 ou 2 frases: o que mudou do kick-off até hoje]

Caio Brayner

Desenvolvedor C

>_ O QUE FIZ

- Estrutura central GameContext / game_state
- Pontuação e alinhamento ético (PI2-90)
- Chefe AURA-67 e sistema de projéteis
- Certificado da US20 e jogar.bat

>_ NÃO CONSEGUI

[Caio preenche]

>_ O QUE APRENDI

[Caio preenche]

Jhorge Araújo

Arquitetura & Lógica Matemática

>_ O QUE FIZ

- Aprovador (A) das frentes de IHC e LMC na matriz RACI
- Consultoria de arquitetura do motor em C
- [Jhorge: completar]

>_ NÃO CONSEGUI

[Jhorge preenche]

>_ O QUE APRENDI

[Jhorge preenche]

João Gabriel

Desenvolvedor Web/C & Arquitetura

>_ O QUE FIZ

- Ranking e estatísticas em C (PI2-177)
- Integração do laço principal (PI2-181)
- Áudio sintetizado (US19) e ícone (US16)
- Integração contínua no GitHub Actions

>_ NÃO CONSEGUI

[João preenche]

>_ O QUE APRENDI

[João preenche]

Julio Cesar

Lógica Matemática & Testador

>_ O QUE FIZ

- Laço de combate e rodadas (PI2-89)
- Motor de digitação e precisão
- Modo Tutorial Guiado (US17)
- Relatório de usabilidade (IHC)

>_ NÃO CONSEGUI

[Julio preenche]

>_ O QUE APRENDI

[Julio preenche]

Larissa Almeida

Lead de Engenharia (FDS)

>_ O QUE FIZ

- 39 casos de teste de sistema (Sprints 01 e 02)
- Screencasts e bug tracker
- Matriz de rastreabilidade
- Seções de versionamento e qualidade do README

>_ NÃO CONSEGUI

[Larissa preenche]

>_ O QUE APRENDI

[Larissa preenche]

Mateus Lacerda

Scrum Master

>_ O QUE FIZ

- Organização do board e dos sprints no Jira
- Seção de implementação do README
- Matriz de duplas e CONTRIBUTING.md
- Estrutura da apresentação final

>_ NÃO CONSEGUI

[Mateus preenche]

>_ O QUE APRENDI

[Mateus preenche]

Matheus Chaves

Design de Interface (IHC)

>_ O QUE FIZ

- Save/load binário com validação (PI2-175)
- Protótipos e storyboards das telas
- Menu estilo terminal e tela 720p (US18)
- Screencast de usabilidade

>_ NÃO CONSEGUI

[Matheus preenche]

>_ O QUE APRENDI

[Matheus preenche]

Theo Monteiro

Requisitos & QA

>_ O QUE FIZ

- Validação lógica da Unidade 2 (PI2-31): 181 verificações
- Catalogação de issues e evidências do Sprint 02
- Relatório de pair programming do Sprint 03
- Slides da apresentação final

>_ NÃO CONSEGUI

[Theo preenche]

>_ O QUE APRENDI

[Theo preenche]

> ALINHAMENTO: 100% // LOCKDOWN ENCERRADO

Obrigado!

Perguntas?

Código: github.com/vdornelass/pi2-squad-e7
Download do jogo: [link da release final]